

Milan George

Gameplay and Systems Programmer | Unreal Engine 5, C++

Bangalore, India | +91 90748 86031 | milankgeorge@gmail.com | mln-grg.github.io | linkedin.com/in/milan-george | github.com/mln-grg

TECHNICAL SKILLS

Languages	C++ (11 and 17), C#, C
Engines	Unreal Engine 5, Unity
Unreal	Mass Entity (ECS and data oriented design), Gameplay Ability System and custom ability frameworks, plugin and module architecture with UnrealBuildTool, World Partition, UMG, Slate and Common UI, navigation and pathfinding, vertex animation for crowd rendering, Niagara, FMOD
Performance	Unreal Insights, CPU, GPU and memory profiling, instanced rendering, per instance LOD tiering, object pooling, garbage collection and object lifetime debugging
Architecture	Data oriented design, ECS, MVVM and data binding, event driven systems, multithreading
Design Patterns	State, Observer, Object Pool, Service Locator, Factory, Command
AI and GenAI	GenAI assisted development, code review and test generation with Claude Code, GitHub Copilot, Cursor and ChatGPT
Tools	Windows and Linux, REST API integration, Git, Plastic SCM, Visual Studio, Rider

PROFESSIONAL EXPERIENCE

Project Lead, Nytwolf Games Bangalore, India | April 2024 to Present

Project Greenleaf (Unreal Engine 5, C++, PC). A Total War style strategy game with large scale real time battles and a campaign overworld.

- Led the battle and overworld layers for a team of five: architecture, systems design, task breakdown and code review, working with designers and artists inside the frame budget.
- Architected soldier simulation in C++ on **Mass Entity**, pairing every entity with a light actor that renders through vertex animation. Sustained **4,000 individually simulated soldiers** in a single battle, with per instance LOD tiers keeping crowd rendering affordable.
- Built a **GAS style ability system** for squads covering 17 combat and traversal abilities, where a player order takes over from whatever the squad chose on its own and the interrupted ability resumes once that order finishes.
- Made a move order expand into a **chain of steps read off the navigation path**, walk, climb a wall, cross a gate, reporting success only once the last step lands.
- Shipped the battle system and a **dynamic MVVM UI framework** as two standalone C++ plugins with their own UnrealBuildTool modules, Blueprint exposed, independently testable and retained as reusable in house technology.
- Built the per soldier animation state machine on **plain C++ state objects instead of UObjects**, one instance per state type shared across all 4,000 soldiers, so a state change allocates nothing and adds no garbage collection work.
- Instrumented the plugin with profiling scopes and used **Unreal Insights** to find where frame time and memory actually went, then fixed it, including a World Partition leak that kept battle cell worlds alive after the return to the campaign map.
- Designed and implemented an **A* grid navigation** and pathing layer in C++ for unit movement, with boids style flocking to hold formation on the move.
- Built a **settlement designer**: grid snapped building placement, and Stronghold style wall building where each piece picks its own form from what it connects to, straight runs, corners, ends and diagonals.

VR Gameplay Programmer, i3 Simulations Bangalore, India | August 2022 to April 2024

- SimGym** (Unreal Engine, PC VR): built a scenario creator with a multigrid system, letting users author detailed simulations without engineering support, inside a fixed VR frame budget.
- ResusVR** (Unity): moved localisation onto a Google Sheet where every term sits beside its translations, filled in through Google Translate and imported in one pass. Builds pull the current text from the sheet at run time, so a wording change no longer needs a new build.

Technical Interviewer and Code Reviewer, Outscal Remote, part time | October 2022 to August 2023

- Ran technical interviews and reviewed C++ and Unity submissions for game programming candidates, giving structured written feedback on architecture, code quality and problem decomposition.

SELECTED PROJECT

Benji 2D, a 2D game engine built from scratch in C++ (github.com/mln-grg/Benji-2D). Signature based ECS with packed component pools, a frame rate independent game loop, a type indexed event bus, AABB collision and a Dear ImGui debug layer.

EDUCATION

B.Tech, Computer Science Engineering, APJ Abdul Kalam Technological University, 2022 | **Full Stack Game Development**, Outscal, 2020 to 2022